Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

List of Teammates

S.No.	Name	College Id	Phone No.
1.	Joelkirubainathan saminathan	joelkirubainathansaminathan23csa@vetias.ac.in	7867040926
2.	Rohit Pranav Devaraj	rohitpranavdevaraj23csa@vetias.ac.in	9361167142
3.	Deeksithasri	deeksithasriss23csa@vetias.ac.in	8667392132
4.	Salomin Reji	salominrejir23csa@vetias.ac.in	6383138173
5.	Vikash	vikashr23csa@vetias.ac.in	93361162522

Title of the Project: -

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

Overview: -

Integrating Zscaler with enterprise Identity Providers (IdPs) like Azure AD, Okta, or Active Directory is a critical step in building a robust Zero Trust architecture. This shift moves security away from traditional, location-based network controls toward an identity-aware model, ensuring that access decisions are based on the verified identity of the user, regardless of their device or physical location.

Technically, the integration relies on SAML for secure authentication and SCIM for automated user lifecycle management. SAML enables the IdP to verify user credentials and provide Zscaler with the assertions necessary to grant access to resources, while SCIM automates the synchronization of user accounts and groups. This real-time data flow ensures that access rights are always current, significantly reducing administrative overhead and ensuring immediate access revocation during offboarding.

From an organizational perspective, this approach establishes a "single source of truth" for identity, allowing for granular, role-based access policies rather than static IP-based rules. This enhances compliance, simplifies auditing, and boosts operational efficiency. Furthermore, by implementing Single Sign-On (SSO), the organization improves the user experience by reducing password fatigue and streamlining the login process across all corporate applications.

The project scope focuses on configuring the Zscaler tenant as a trusted Service Provider and establishing a secure trust relationship with the enterprise IdP. By validating the authentication flow and the synchronization of user metadata, the organization achieves a scalable and highly secure framework that effectively protects a distributed workforce while maintaining a seamless, policy-driven user experience.

Abstract: -

In the contemporary landscape of hybrid work and cloud-first strategies, traditional network-centric security models are increasingly insufficient. As organizations transition away from on-premises data centres to distributed cloud environments, the reliance on IP-based security boundaries creates significant vulnerabilities. This project details the strategic integration of Zscaler's cloud-native security platform with enterprise Identity Providers (IdPs) such as Microsoft Azure AD, Okta, and Active Directory. The core objective is to shift the security paradigm toward a "Zero Trust" architecture, where access to enterprise resources is predicated on verified user identity and context rather than network location.

Technically, the integration leverages industry-standard protocols to create a seamless security ecosystem. Security Assertion Markup Language (SAML) is employed to handle secure, centralized authentication, allowing Zscaler to act as a Service Provider that trusts the IdP's identity verification. Simultaneously, the System for Cross-domain Identity Management (SCIM) is implemented to automate the synchronization of user identities, group memberships, and lifecycle attributes. This automated data exchange ensures that the security platform possesses real-time visibility into the organization's user base, facilitating immediate access revocation during offboarding and minimizing the risk of "orphan" accounts.

Beyond the technical implementation, this project addresses significant operational and security challenges. By centralizing identity management, the organization establishes a single "source of truth," enabling security administrators to enforce granular, role-based access control (RBAC) policies. This replaces complex, unmanageable IP-based access lists with user-centric security policies that can follow the employee across various corporate applications and locations. Furthermore, the deployment of Single Sign-On (SSO) directly contributes to improved productivity and end-user satisfaction by consolidating disparate login processes into a single, unified experience.

Ultimately, this integration provides a scalable, resilient defence mechanism capable of protecting a modern, distributed workforce. By aligning Zscaler's advanced threat prevention capabilities with the organizational identity store, this project not only enhances the security posture against unauthorized access but also simplifies compliance, strengthens audit trails, and provides a robust foundation for future security initiatives. This framework represents a critical investment in securing the enterprise perimeter in an era of borderless connectivity.

Use Case Scenarios: -

Use Case Scenario 1: Secure Contractor Access

A third-party contractor is hired to manage a specific set of development servers for a limited period. Instead of giving them broad network access, the organization assigns them to a "Contractor" group in the IdP. Zscaler uses this group membership to restrict their access only to the necessary development infrastructure, ensuring the rest of the network remains invisible to them.

Output:

- User identity verified: Contractor_Group.
- Connection restricted: Access limited exclusively to Development_Servers (Dev_Zone).
- Security enforcement: Strict monitoring of session activity for audit compliance.
- No lateral movement: Blocked from accessing other internal systems like Finance or HR.
- Access expiry: Automatic termination of access when the account is deactivated in the IdP.

Use Case Scenario 2: Automated Employee Provisioning

When a new employee joins the organization, they are added to the corporate IdP (e.g., Azure AD). The integration ensures that the user is immediately synchronized into the Zscaler platform without IT having to manually create accounts in multiple systems, ensuring a secure and efficient onboarding process from day one.

Output:

- Identity synchronization: SCIM-based automatic user provisioning.
- Role assignment: User automatically inherits policies associated with their department group.
- Scalability: Immediate creation of security profiles for large batches of new hires.
- Security consistency: Ensures every new user is bound by default security policies upon account creation.
- Audit trail: Centralized log of account activation linked to the source IdP.

Use Case Scenario 3: SaaS Application Security (Shadow IT Control)

Employees often use unsanctioned SaaS applications that can create security vulnerabilities. By integrating Zscaler with the IdP, the organization enforces that any user attempting to access a SaaS application must first authenticate through the enterprise IdP. This forces all traffic through the security gateway, effectively eliminating "Shadow IT."

Output:

- Identity verification: Mandatory IdP authentication required for all SaaS app access.
- Policy enforcement: Unauthorized or risky SaaS apps are blocked at the cloud gateway.
- Visibility: Admin dashboard identifies all users attempting to access unapproved applications.
- Data protection: Prevents corporate data from being uploaded to non-sanctioned personal clouds.
- Compliance: Ensures all SaaS usage aligns with enterprise security standards.

Use Case Scenario 4: Instant Emergency Offboarding

When an employee’s employment is terminated, the organization needs to ensure that access to all internal and cloud-based systems is revoked immediately. By disabling the user's account in the central IdP, the integration triggers an automatic and near-instant signal to Zscaler to block all active connections for that user across the entire enterprise.

Output:

- Immediate revocation: Real-time synchronization of "Account Disabled" status via SCIM.
- Session termination: All active Zscaler-protected sessions are killed instantly.
- Security lockout: User is blocked from all web gateways, private apps, and SaaS platforms.
- Risk mitigation: Zero-day gap between termination and access revocation.
- Compliance reporting: Automated log entry confirming successful access denial post-termination.

Use Case Scenario 5: Granular Role-Based Access for Financial Applications

A company requires that only authorized members of the Finance department can access sensitive internal ledger applications. By utilizing IdP group claims, Zscaler enforces a policy

where every connection request is checked against the user’s specific department attribute, ensuring only the right people access the right financial data.

Output:

- Identity validated: Attribute-based group check (member of: "Finance_Dept").
- Granular enforcement: Non-Finance users are denied access even if they discover the application URL.
- Policy precision: Security rules move with the user, regardless of their network connection.
- Audit readiness: Detailed logs showing exactly which Finance user accessed which ledger tool.
- Consistent user experience: Single Sign-On (SSO) enables seamless login without password re-entry.

Stage – 1

Title of the Project: -

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

List of Vulnerability Table: -

S.No.	Vulnerability Name	CWE/CVE - No
1.	Active Debug Code	CWE-489
2.	Cleartext Storage in a File or on Disk	CWE-313
3.	Cleartext Storage in a File or on Disk	CWE-112
4.	Origin Validation Error	CWE-346
5.	Use of Weak Hash	CWE-328
6.	Configuration	CWE-16
7.	Relative Path Traversal	CWE-23
8.	Improper Authentication	CWE-287
9.	Untrusted Search Path	CWE-426
10.	SQL Injection: Hibernate	CWE-564

REPORT: -

1. **Vulnerability Name: -** Active Debug Code

   CWE/CVE: - CWE-489

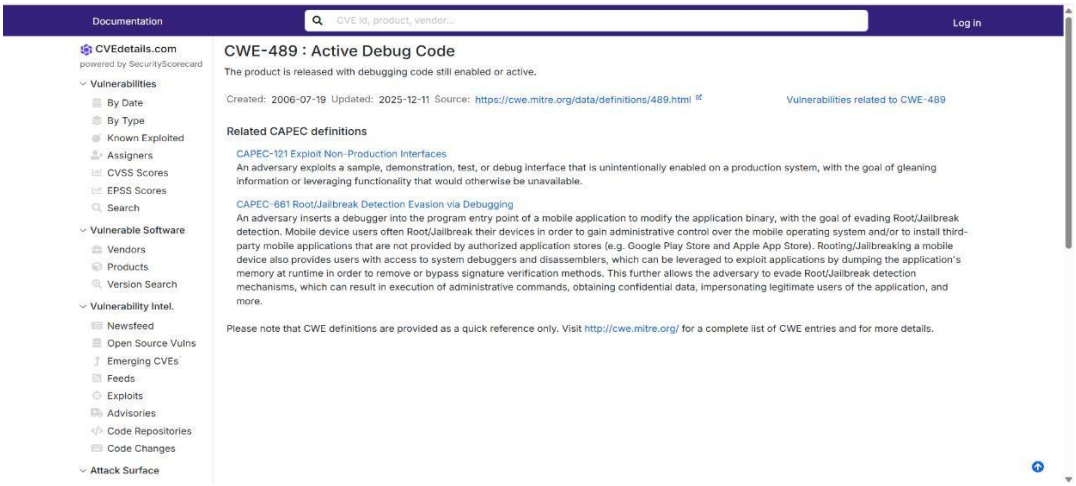
   OWASP/SANS Category: - A02:2025 Security Misconfiguration

   Description: -

Active Debug Code (CWE-489) occurs when debugging statements, test functions, or developer-related code remain enabled in a production environment. These debugging mechanisms are normally used during development to test and troubleshoot applications, but if they are not removed before deployment, they can expose sensitive internal information. Debug code may reveal system paths, stack traces, configuration details, application logic, or even test credentials.

Business Impact: -

The presence of active debug code can lead to significant security risks for an organization. Attackers may exploit the exposed debug information to gain insights into the application's internal structure, making it easier to identify and exploit other vulnerabilities. This can lead to information disclosure, unauthorized access, or even system compromise. Additionally, exposure of sensitive technical details may damage the organization's reputation, reduce customer trust, and potentially result in regulatory or compliance violations if confidential data is leaked.



2. **Vulnerability Name: -** Cleartext Storage in a File or on Disk

   CWE/CVE: - CWE-313

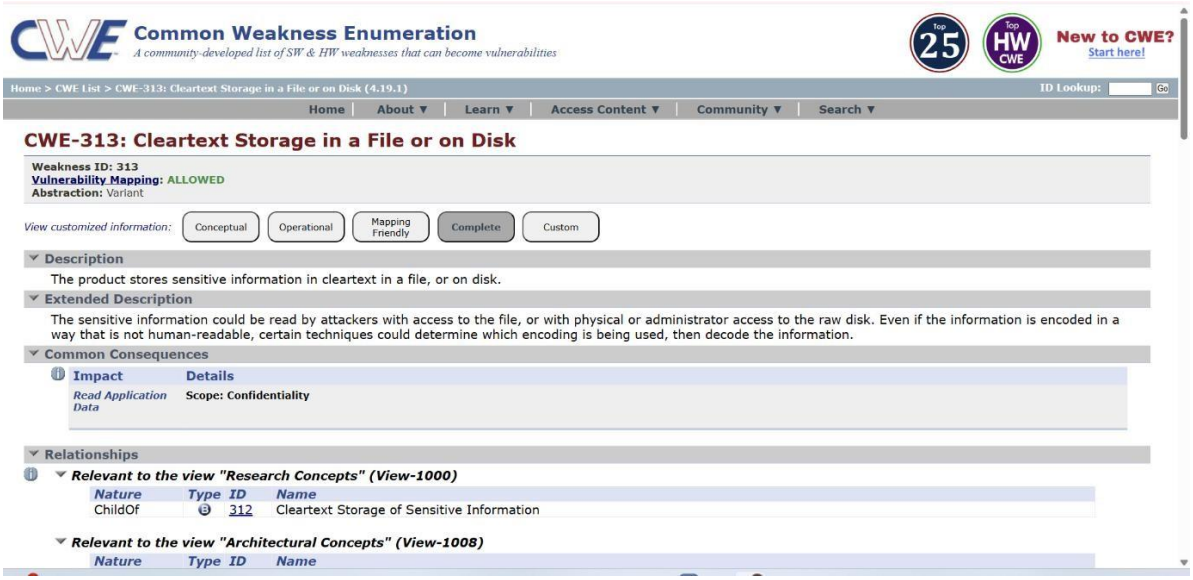
   OWASP/SANS Category: - A06:2025 Insecure Design

   Description: -

Cleartext Storage in a File or on Disk (CWE-313) occurs when sensitive information such as passwords, authentication tokens, personal data, or confidential system details are stored on a system in plain text without encryption or proper protection. When data is stored in cleartext, anyone with access to the file system—whether an attacker, malicious insider, or compromised application—can easily read and misuse the information. This vulnerability is often related to poor security architecture and is associated with OWASP Top 10 A06:2025 Insecure Design, where secure design principles such as data protection and encryption are not properly implemented.

   Business Impact: -

If sensitive data is stored in cleartext on disk or in files, it can be easily accessed and exploited by attackers who gain system access. This may lead to exposure of user credentials, confidential business data, or personally identifiable information (PII). Such breaches can result in unauthorized access to systems, financial losses, identity theft, and misuse of critical organizational data. Additionally, organizations may face serious reputational damage, loss of customer trust, and legal or regulatory penalties due to failure to properly protect sensitive information.



3. **Vulnerability Name: -** Missing XML Validation

   CWE/CVE: - CWE-112

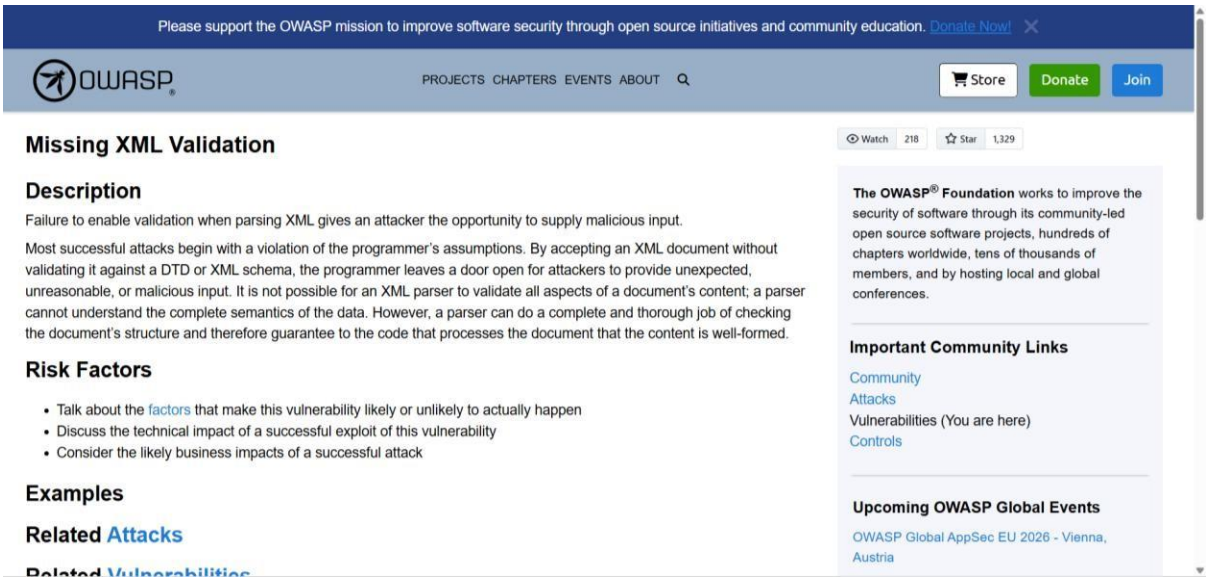
   OWASP/SANS Category: - A05:2025 Injection

   Description: -

Missing XML Validation (CWE-112) occurs when an application processes XML input without properly validating its structure, format, or content against a defined schema or validation rules. Without proper validation, attackers can manipulate XML data to include malicious payloads, unexpected elements, or malformed structures. This may allow attackers to exploit the application through injection attacks, unauthorized data manipulation, or system misuse. Such weaknesses are commonly associated with OWASP Top 10 A05:2025 Injection because unvalidated input can be used to inject malicious commands or data into the system.

   Business Impact: -

If XML inputs are not properly validated, attackers may exploit the application to inject malicious data, manipulate application logic, or gain unauthorized access to sensitive information. This can lead to data breaches, system compromise, or unauthorized modification of business data. Additionally, exploitation of this vulnerability can disrupt application functionality and services, resulting in operational downtime and financial losses. Organizations may also suffer reputational damage, loss of customer trust, and potential legal or regulatory consequences if sensitive data is exposed or compromised.



4. **Vulnerability Name: -** Origin Validation Error

   CWE/CVE: - CWE-346

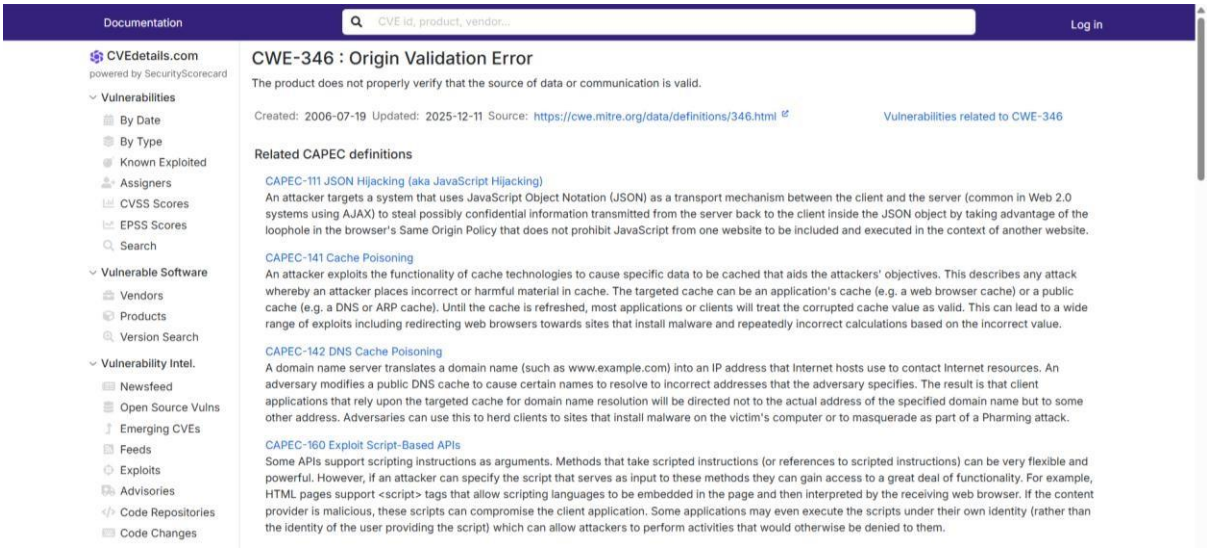
   OWASP/SANS Category: - A07:2025 Authentication Failures

   Description: -

Origin Validation Error (CWE-346) occurs when an application fails to properly verify the origin of a request before processing it. This means the system does not correctly check whether the request is coming from a trusted domain, application, or user session. Without proper origin validation mechanisms such as checking the **Origin** or **Referrer** headers, attackers may craft malicious requests that appear legitimate. This vulnerability is often linked to OWASP Top 10 A07:2025 Authentication Failures because it can allow attackers to bypass authentication or authorization controls and perform actions on behalf of a legitimate user.

   Business Impact: -

If origin validation is not properly implemented, attackers may exploit the system to perform unauthorized actions, manipulate user sessions, or access sensitive resources. This could allow attackers to impersonate legitimate users, perform unauthorized transactions, or alter critical application data. Such exploitation may lead to data breaches, financial losses, and disruption of business operations. Additionally, organizations may suffer reputational damage, loss of customer trust, and potential legal or regulatory consequences if unauthorized activities compromise sensitive information.



5. **Vulnerability Name: -** Use of Weak Hash

   CWE/CVE: - CWE-328

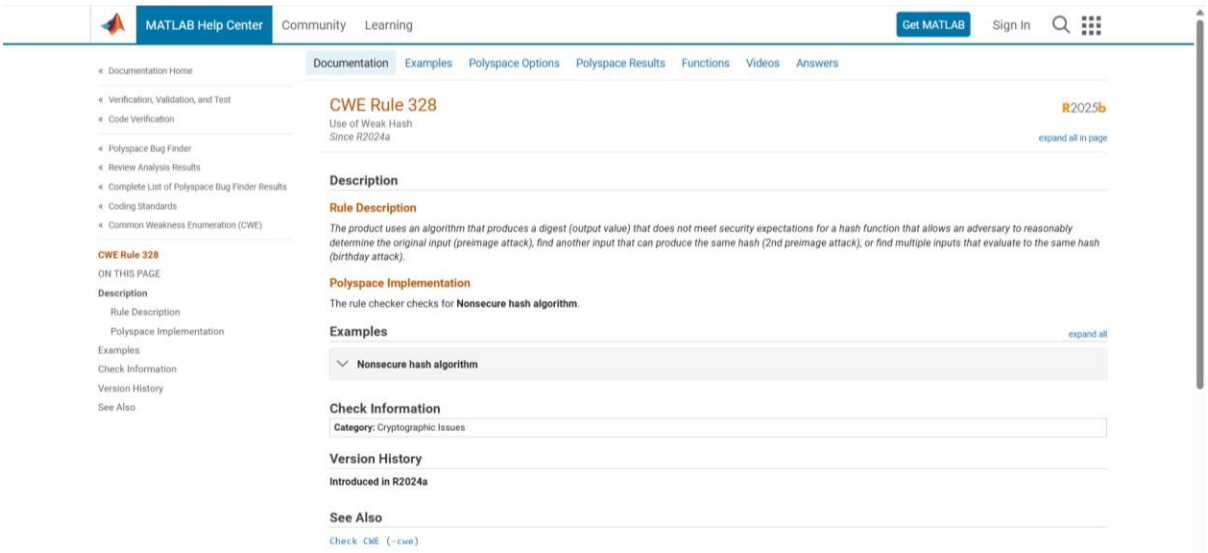
   OWASP/SANS Category: - A04:2025 Cryptographic Failures

   Description: -

Use of Weak Hash (CWE-328) occurs when an application relies on outdated or insecure hashing algorithms such as MD5 or SHA-1 to protect sensitive information like passwords, digital signatures, or integrity checks. Weak hashing algorithms are vulnerable to cryptographic attacks, including collision attacks and brute-force attempts, which can allow attackers to reverse or manipulate hashed data. Because these algorithms no longer provide strong security protection, their use represents poor cryptographic implementation and is categorized under OWASP Top 10 A04:2025 Cryptographic Failures.

   Business Impact: -

If weak hashing algorithms are used to protect sensitive information, attackers may be able to crack hashed passwords, forge digital signatures, or manipulate data integrity checks. This can lead to unauthorized access to user accounts, exposure of confidential data, and compromise of critical systems. Such incidents may result in financial losses, operational disruption, and damage to the organization's reputation. Additionally, failure to implement strong cryptographic practices may lead to non-compliance with security standards and regulatory requirements, potentially resulting in legal penalties and loss of customer trust.



6. **Vulnerability Name: -** Configuration

   CWE/CVE: - CWE-16

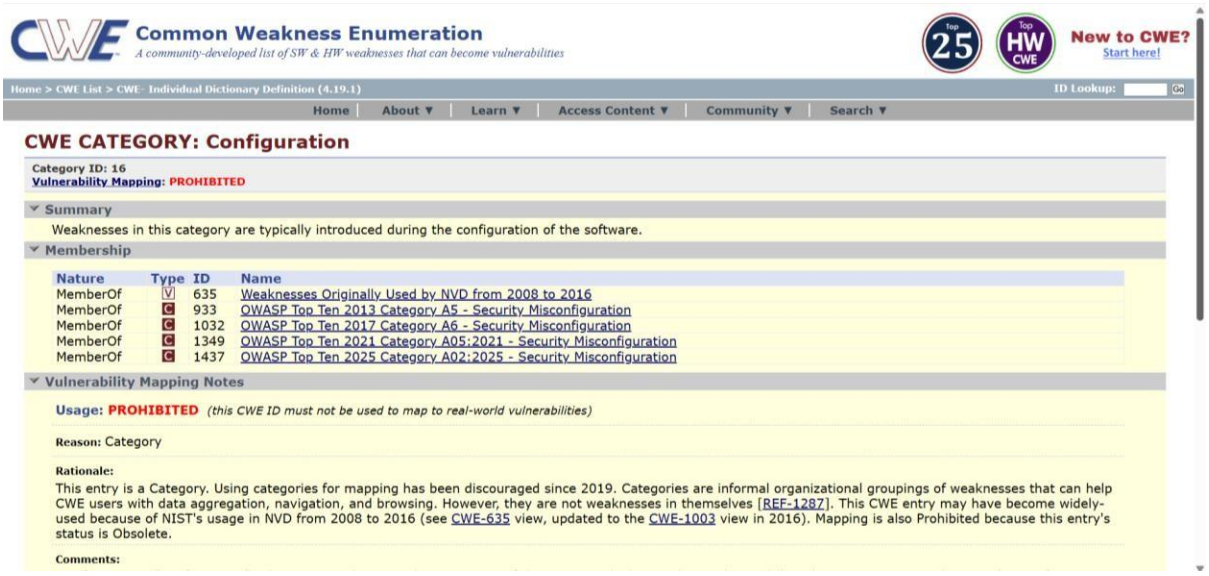
   OWASP/SANS Category: - A02:2025 Security Misconfiguration

   Description: -

Use of Weak Hash (CWE-328) occurs when an application relies on outdated or insecure hashing algorithms such as MD5 or SHA-1 to protect sensitive information like passwords, digital signatures, or integrity checks. Weak hashing algorithms are vulnerable to cryptographic attacks, including collision attacks and brute-force attempts, which can allow attackers to reverse or manipulate hashed data. Because these algorithms no longer provide strong security protection, their use represents poor cryptographic implementation and is categorized under OWASP Top 10 A04:2025 Cryptographic Failures.

   Business Impact: -

If weak hashing algorithms are used to protect sensitive information, attackers may be able to crack hashed passwords, forge digital signatures, or manipulate data integrity checks. This can lead to unauthorized access to user accounts, exposure of confidential data, and compromise of critical systems. Such incidents may result in financial losses, operational disruption, and damage to the organization's reputation. Additionally, failure to implement strong cryptographic practices may lead to non-compliance with security standards and regulatory requirements, potentially resulting in legal penalties and loss of customer trust.



7. **Vulnerability Name: -** Relative Path Traversal

   CWE/CVE: - CWE-23

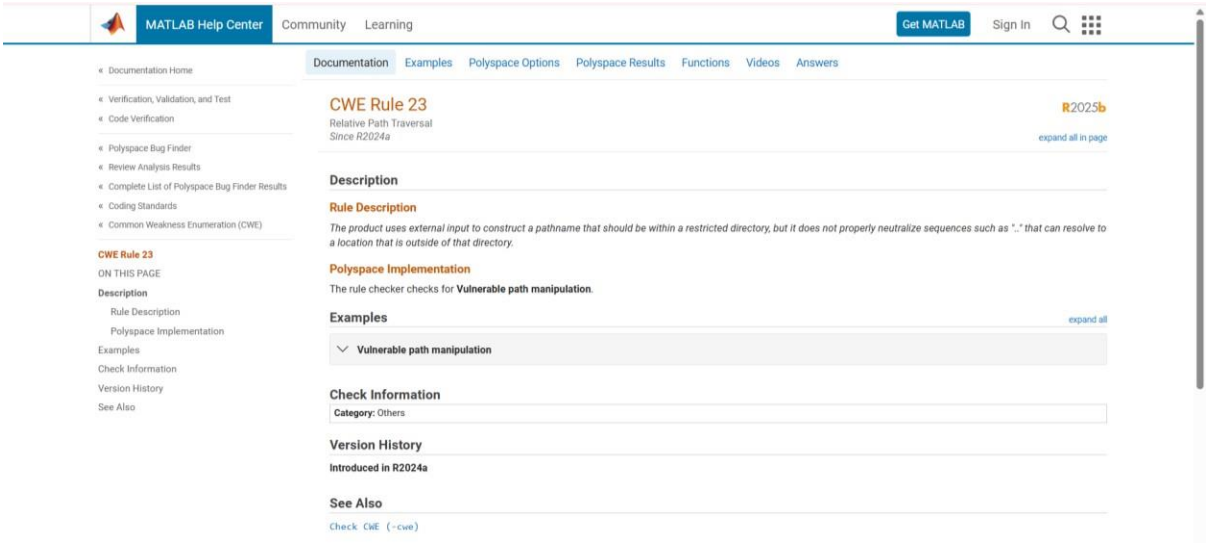
   OWASP/SANS Category: - A01:2025 Broken Access Control

   Description: -

Relative Path Traversal (CWE-23) occurs when an application allows user input to manipulate file paths without proper validation or sanitization. Attackers can exploit this vulnerability by using special characters such as ../ to move up directories and access files or directories outside the intended location. This can allow unauthorized access to sensitive system files, configuration files, or application resources. Such vulnerabilities are commonly associated with OWASP Top 10 A01:2025 Broken Access Control because they bypass intended access restrictions and allow users to access resources that should normally be protected.

Business Impact: -

If Relative Path Traversal is successfully exploited, attackers may gain unauthorized access to sensitive files such as configuration files, application source code, or system credentials. This exposure can lead to further attacks, including privilege escalation, data theft, or complete system compromise. The vulnerability may also allow attackers to modify or delete critical files, disrupting application functionality and business operations. As a result, organizations may experience financial losses, reputational damage, and potential legal or regulatory consequences due to the exposure of sensitive information.



8. **Vulnerability Name: -** Improper Authentication

CWE/CVE: - CWE-287

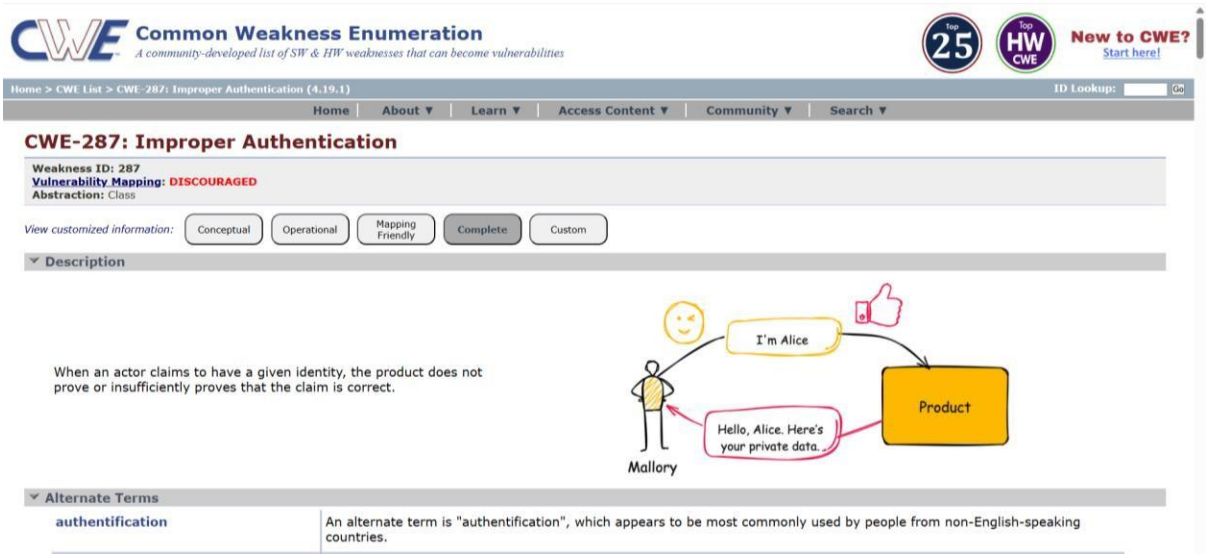
OWASP/SANS Category: - A07:2025 Authentication Failures

Description: -

Improper Authentication (CWE-287) occurs when an application fails to correctly verify the identity of a user, system, or process before granting access to resources or functionalities. This may happen due to weak authentication mechanisms, missing authentication checks, improper session management, or flawed login implementations. As a result, attackers may bypass authentication controls and gain unauthorized access to restricted areas of the system. This vulnerability is categorized under OWASP Top 10 A07:2025 Authentication Failures, as it involves weaknesses in verifying and managing user identities.

Business Impact: -

If improper authentication vulnerabilities are exploited, attackers may gain unauthorized access to user accounts, administrative functions, or sensitive business data. This could lead to data breaches, manipulation of application data, or unauthorized transactions. In severe cases, attackers may take full control of the system or impersonate legitimate users to perform malicious activities. Such incidents can result in financial losses, operational disruptions, reputational damage, and potential legal or regulatory consequences due to compromised security and exposure of sensitive information.



9. **Vulnerability Name: -** Untrusted Search Path

CWE/CVE: - CWE-426

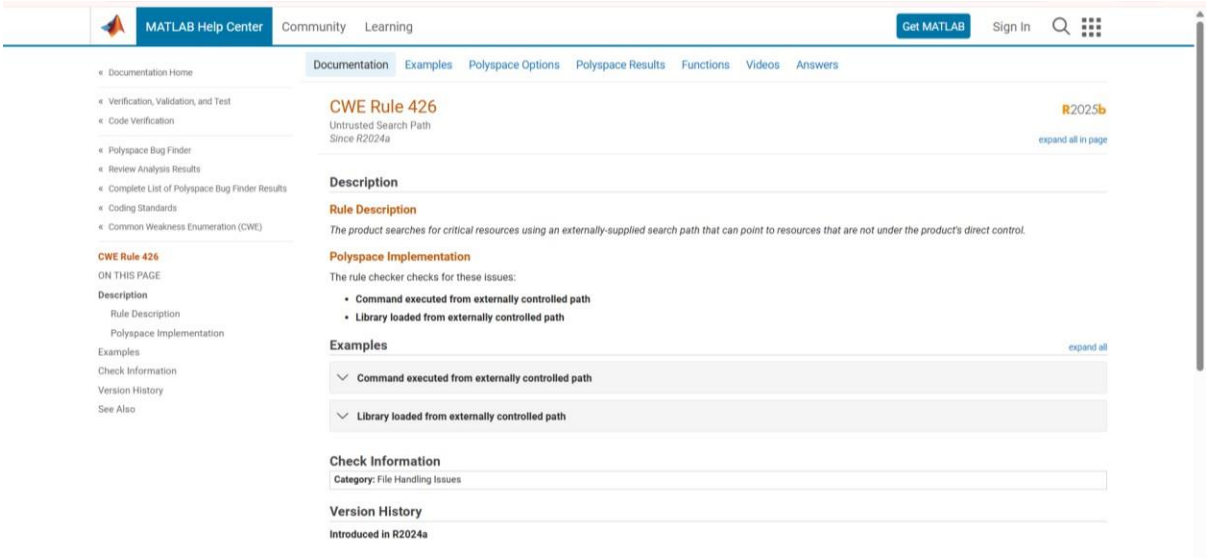
OWASP/SANS Category: - A08:2025 Software or Data Integrity Failures

Description: -

Untrusted Search Path (CWE-426) occurs when an application searches for and loads executable files, libraries, or resources from directories that are not properly secured or trusted. If the system includes directories that can be modified by attackers in its search path, a malicious file with the same name as a legitimate one may be executed instead. This can allow attackers to run unauthorized code or manipulate the behavior of the application. This vulnerability is associated with OWASP Top 10 A08:2025 Software or Data Integrity Failures because it can compromise the integrity of software and system operations.

Business Impact: -

If an untrusted search path vulnerability is exploited, attackers may place malicious files in accessible directories that the application searches first. As a result, the system may execute the malicious code instead of the intended legitimate program or library. This can lead to unauthorized system access, malware execution, data manipulation, or complete system compromise. Such incidents may disrupt business operations, cause financial losses, and damage the organization's reputation. Additionally, compromised software integrity may result in regulatory compliance issues and loss of customer trust.



10. **Vulnerability Name: -** SQL Injection: Hibernate

CWE/CVE: - CWE-564

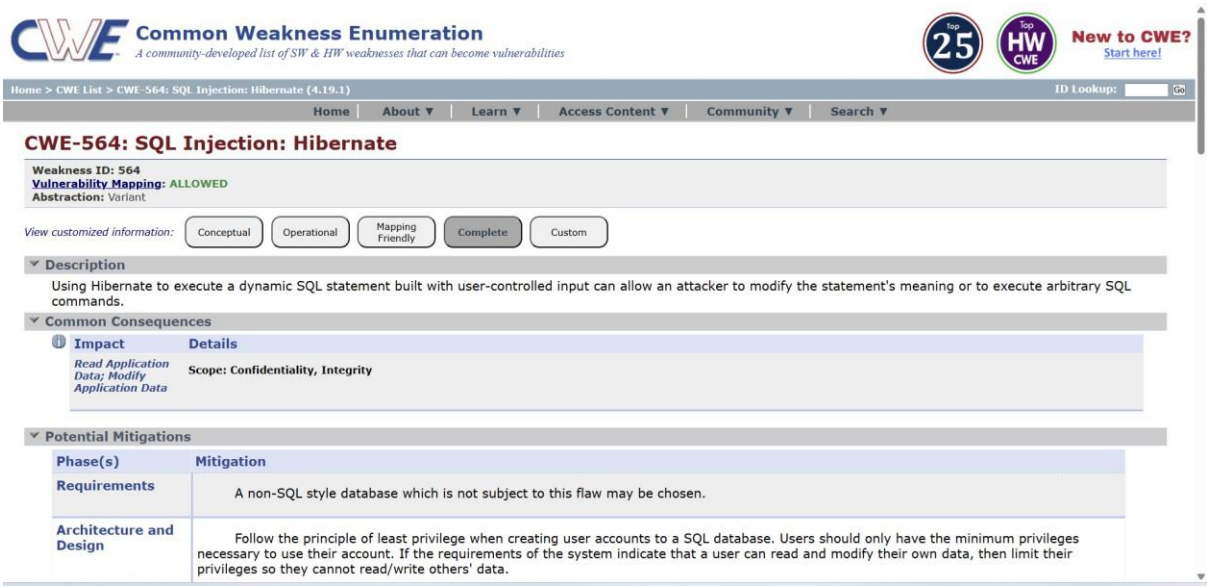
OWASP/SANS Category: - A05:2025 Injection

Description: -

SQL Injection: Hibernate (CWE-564) occurs when user-supplied input is improperly handled within Hibernate Query Language (HQL) queries, allowing attackers to inject malicious SQL or HQL commands into the application's database queries. If the application dynamically constructs queries without proper parameterization or input validation, attackers may manipulate the query structure to access, modify, or delete data from the database. This vulnerability falls under OWASP Top 10 A05:2025 Injection because it involves injecting malicious commands into a query that is executed by the database.

Business Impact: -

If SQL Injection vulnerabilities in Hibernate are exploited, attackers may gain unauthorized access to sensitive database information such as user credentials, financial records, or confidential business data. They may also modify or delete critical data, leading to data corruption or loss. In severe cases, attackers can bypass authentication mechanisms, escalate privileges, or completely compromise the database server. This can result in financial losses, operational disruption, reputational damage, and legal or regulatory consequences due to exposure or manipulation of sensitive information.



Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

Home > CWE List > CWE-564: SQL Injection: Hibernate (4.19.1) | ID Lookup: Go

Home | About ▼ | Learn ▼ | Access Content ▼ | Community ▼ | Search ▼

CWE-564: SQL Injection: Hibernate

Weakness ID: 564
Vulnerability Mapping: ALLOWED
Abstraction: Variant

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Using Hibernate to execute a dynamic SQL statement built with user-controlled input can allow an attacker to modify the statement's meaning or to execute arbitrary SQL commands.

Common Consequences

Impact	Details
Read Application Data; Modify Application Data	Scope: Confidentiality, Integrity

Potential Mitigations

Phase(s)	Mitigation
Requirements	A non-SQL style database which is not subject to this flaw may be chosen.
Architecture and Design	Follow the principle of least privilege when creating user accounts to a SQL database. Users should only have the minimum privileges necessary to use their account. If the requirements of the system indicate that a user can read and modify their own data, then limit their privileges so they cannot read/write others' data.

Stage 2

Overview: -

Nessus is a powerful and widely used vulnerability assessment tool designed to help organizations identify security weaknesses in their IT infrastructure. It is developed and maintained by **Tenable**, a leading cybersecurity company known for providing advanced security solutions. Nessus is primarily used by security professionals, system administrators, and ethical hackers to scan networks, systems, and applications for vulnerabilities that could potentially be exploited by cyber attackers. By detecting these weaknesses early, organizations can take corrective actions to strengthen their overall security posture and protect sensitive data.

Nessus works by performing automated vulnerability scans using a vast library of vulnerability checks known as plugins. These plugins allow the tool to identify thousands of known vulnerabilities, including outdated software versions, missing security patches, weak system configurations, and known exploits. During the scanning process, Nessus sends various requests to the target systems and analyzes their responses to determine whether any vulnerabilities exist. The tool then evaluates the detected issues and categorizes them according to severity levels such as critical, high, medium, low, and informational. This classification helps security teams prioritize which vulnerabilities need immediate attention.

Another important capability of Nessus is its support for both authenticated and unauthenticated scanning methods. In an authenticated scan, the scanner logs into the target system using provided credentials, allowing it to perform deeper inspection and identify vulnerabilities that may not be visible from outside the system. On the other hand, unauthenticated scans simulate an external attacker’s perspective by scanning systems without login access. This dual approach provides a more comprehensive assessment of system security and helps organizations understand both internal and external security risks.

In addition to vulnerability detection, Nessus also provides detailed reporting and remediation guidance. After completing a scan, it generates comprehensive reports that include descriptions of each vulnerability, the potential risk it poses, and recommended solutions to fix the issue. These reports help security teams understand the security status of their infrastructure and take appropriate actions to mitigate threats. Nessus also regularly updates its vulnerability database and plugins to ensure it can detect the latest security threats and emerging vulnerabilities.

Overall, Nessus plays a crucial role in modern cybersecurity practices by enabling organizations to proactively identify and manage vulnerabilities. Its powerful scanning capabilities, extensive vulnerability coverage, detailed reporting features, and user-friendly interface make it one of the most trusted vulnerability scanning tools used across industries. By integrating Nessus into their security strategy, organizations can continuously monitor their systems for weaknesses and significantly reduce the risk of cyberattacks.

Target Website: http://zero.webappsecurity.com/

Target IP Address: **54.82.22.214**

List of Vulnerabilities: -

S.No.	Vulnerability Name	Severity	Plugins
1.	SSL Version 2 and 3 Protocol Detection	Critical	20007
2.	SSL Medium Strength Cipher Suites Supported (SWEET32)	High	42873
3.	JQuery 1.2 < 3.5.0 Multiple XSS	Medium	136929
4.	SSL DROWN Attack Vulnerability (Decrypting RSA with Obsolete and Weakened encryption)	Medium	89058
5.	OpenSSL SSL_OP_NETSCAPE_REUSE_CIPHER_CHANGE_BUG Session Resume Ciphersuite Downgrade Issue	Medium	51892
6.	SSL Certificate Cannot Be Trusted	Medium	51192
7.	TLS Version 1.0 Protocol Detection	Medium	104743
8.	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)	Low	83875
9.	SSLv3 Padding Oracle On Downgraded Legacy Encryption Vulnerability (POODLE)	Low	78479
10.	SSL/TLS EXPORT_DHE <= 512-bit Export Cipher Suites Supported (Logjam)	Low	83738

REPORT

1. **Vulnerability Name: -** SSL Version 2 and 3 Protocol Detection

   severity: - Critical

   Plugin: - 20007

   Description: - The remote service accepts connections encrypted using SSL 2.0 and/or SSL 3.0. These versions of SSL are affected by several cryptographic flaws, including:

   - An insecure padding scheme with CBC ciphers.
   - Insecure session renegotiation and resumption schemes.

   An attacker can exploit these flaws to conduct man-in-the-middle attacks or to decrypt communications between the affected service and clients.

   Although SSL/TLS has a secure means for choosing the highest supported version of the protocol (so that these versions will be used only if the client or server support nothing better), many web browsers implement this in an unsafe way that allows an attacker to downgrade a connection (such as in POODLE). Therefore, it is recommended that these protocols be disabled entirely.

   NIST has determined that SSL 3.0 is no longer acceptable for secure communications. As of the date of enforcement found in PCI DSS v3.1, any version of SSL will not meet the PCI SSC's definition of 'strong cryptography'.

   Solution: - To mitigate this vulnerability, organizations should disable SSL 2.0 and SSL 3.0 protocols on all servers and network devices and only allow secure versions of the TLS protocol (TLS 1.2 or TLS 1.3).

   System administrators should update the server configuration to remove support for outdated SSL versions. For example, in web servers such as Apache, Nginx, or IIS, administrators should explicitly configure the

system to accept only secure TLS protocols. Additionally, weak cipher suites associated with SSL should be removed.

All systems, applications, and libraries should be updated to the latest security patches to ensure compatibility with modern TLS standards. After making configuration changes, organizations should perform security scans and vulnerability assessments (for example using vulnerability scanners like Nessus) to verify that SSL 2.0 and SSL 3.0 have been completely disabled.

Organizations should also implement strong encryption policies, ensure regular security audits, and follow industry security guidelines such as NIST recommendations and PCI DSS requirements to maintain secure communications.

Business Impact: - Enabling SSL 2.0 and SSL 3.0 protocols exposes the system to serious security risks because these protocols contain known cryptographic weaknesses. Attackers can exploit these weaknesses to intercept or manipulate secure communications between users and the server.

One major risk is a Man-in-the-Middle (MITM) attack, where an attacker secretly intercepts the communication between a client and the server. By exploiting vulnerabilities such as the POODLE attack, attackers can force a downgrade from secure TLS protocols to SSL 3.0 and potentially decrypt sensitive information.

This vulnerability may lead to data breaches involving confidential data such as login credentials, personal information, or financial details. It also affects regulatory compliance, since standards such as PCI DSS and NIST no longer consider SSL protocols secure. Organizations that continue using SSL may face compliance violations, financial penalties, reputational damage, and loss of customer trust.

If exploited, attackers may gain unauthorized access to sensitive communications, which can compromise the overall security posture of the organization.

2. **Vulnerability Name: -** SSL Medium Strength Cipher Suites Supported (SWEET32)

   Severity: - High

   Plugin: - 42873

   Description: - The remote host supports the use of SSL ciphers that offer medium strength encryption. Nessus regards medium strength as any encryption that uses key lengths at least 64 bits and less than 112 bits, or else that uses the 3DES encryption suite.

   Note that it is considerably easier to circumvent medium strength encryption if the attacker is on the same physical network.

   Solution: - To mitigate this vulnerability, administrators should disable medium-strength SSL/TLS cipher suites, particularly those using 3DES encryption, on all servers and network devices.

   Servers should be configured to allow only strong cryptographic cipher suites, such as those using AES (Advanced Encryption Standard) with 128-bit or 256-bit keys and modern TLS protocols like TLS 1.2 or TLS 1.3. Weak and outdated ciphers should be removed from the server configuration.

   System administrators should also ensure that web servers, applications, and SSL/TLS libraries are updated to the latest versions so they support modern secure encryption standards. After implementing the changes, organizations should perform security testing and vulnerability scanning using tools such as Nessus to confirm that weak cipher suites are no longer supported.

   Implementing strong encryption policies, regularly reviewing SSL/TLS configurations, and following recommended guidelines from security standards organizations will help ensure that secure communication channels are maintained.

   Business Impact: - The presence of medium-strength SSL cipher suites, particularly those using 3DES encryption, exposes the system to security risks such as the SWEET32 attack. These cipher suites use shorter block sizes (64-bit), which makes them vulnerable to collision attacks when large amounts of encrypted traffic are processed.

An attacker who is positioned on the same network as the victim may capture large volumes of encrypted data and exploit the weakness in the 64-bit block cipher to recover sensitive information. This could allow attackers to decrypt session cookies, authentication tokens, or other confidential data transmitted between users and the server.

Exploitation of this vulnerability could lead to unauthorized access to user sessions, exposure of sensitive business data, and compromise of confidential communications. Additionally, continuing to support weak cipher suites may violate security best practices and compliance standards, potentially affecting regulatory compliance and damaging the organization's reputation.

3. **Vulnerability Name: -** JQuery 1.2 < 3.5.0 Multiple XSS

   Severity: - Medium

   Plugin: - 136929

   Description: - According to the self-reported version in the script, the version of JQuery hosted on the remote web server is greater than or equal to 1.2 and prior to 3.5.0. It is, therefore, affected by multiple cross site scripting vulnerabilities.

   Note, the vulnerabilities referenced in this plugin have no security impact on PAN-OS, and/or the scenarios required for successful exploitation do not exist on devices running a PAN-OS release.

   Solution: - To remediate this vulnerability, the jQuery library should be upgraded to version 3.5.0 or later, as newer versions contain fixes for the identified XSS vulnerabilities.

   Developers should review the web application and replace the outdated jQuery version with the latest stable release. All dependent scripts and plugins should also be tested to ensure compatibility with the updated version.

   Additionally, organizations should implement secure coding practices such as proper input validation, output encoding, and Content Security Policy (CSP) to reduce the risk of XSS attacks.

Regular patch management, vulnerability scanning, and software dependency updates should be performed to ensure that third-party libraries remain secure and up to date.

Business Impact: - The web application is using a jQuery version between 1.2 and 3.5.0, which contains multiple Cross-Site Scripting (XSS) vulnerabilities. These vulnerabilities may allow attackers to inject malicious scripts into web pages that are viewed by other users.

If successfully exploited, attackers can execute malicious JavaScript in a user's browser. This may lead to session hijacking, theft of authentication cookies, redirection to malicious websites, or unauthorized actions performed on behalf of legitimate users. Such attacks can compromise user accounts and expose sensitive information.

Although the plugin notes that the vulnerability may not directly impact PAN-OS devices, using outdated libraries still increases the overall attack surface of the web application. If exploited in other environments, it could lead to data leakage, application manipulation, and reputational damage to the organization.

4. **Vulnerability Name: -** SSL DROWN Attack Vulnerability (Decrypting RSA with Obsolete and Weakened encryption)

   Severity: - Medium

   Plugin: - 89058

   Description: - The remote host supports SSLv2 and therefore may be affected by a vulnerability that allows a cross-protocol Breitenbach padding oracle attack known as DROWN (Decrypting RSA with Obsolete and Weakened Encryption). This vulnerability exists due to a flaw in the Secure Sockets Layer Version 2 (SSLv2) implementation, and it allows captured TLS traffic to be decrypted. A man-in-the-middle attacker can exploit this to decrypt the TLS connection by utilizing previously captured traffic and weak cryptography along with a series of specially crafted connections to an SSLv2 server that uses the same private key.

Solution: - To remediate this vulnerability, administrators should completely disable SSLv2 on all servers and network devices. Modern secure protocols such as TLS 1.2 or TLS 1.3 should be used instead.

If the same SSL certificate or private key is used on multiple systems, ensure that no system supporting SSLv2 uses the same key. If SSLv2 was previously enabled, it is recommended to replace the existing TLS certificate and generate a new private key to eliminate the risk of key compromise.

Server configurations should also be updated to remove weak cipher suites and enforce strong encryption standards. After implementing the changes, a vulnerability scan using tools like Nessus should be performed to verify that SSLv2 is fully disabled.

Regular security updates, strong encryption policies, and periodic vulnerability assessments should be maintained to prevent similar cryptographic vulnerabilities in the future.

Business Impact: - This vulnerability occurs because the remote server supports SSLv2, an outdated and insecure protocol. The presence of SSLv2 makes the system vulnerable to the DROWN (Decrypting RSA with Obsolete and Weakened Encryption) attack, which allows attackers to decrypt secure TLS communications.

In this attack, a Man-in-the-Middle (MITM) attacker can capture encrypted TLS traffic between a user and the server. By exploiting weaknesses in the SSLv2 protocol and using specially crafted requests, the attacker can eventually decrypt the captured data if the same private key is used on an SSLv2-enabled server.

Successful exploitation may lead to exposure of sensitive information such as login credentials, session cookies, confidential business data, or personal user information. This can result in data breaches, unauthorized access, financial loss, and reputational damage to the organization. Additionally, the use of outdated protocols like SSLv2 may lead to non-compliance with modern security standards and regulations.

5. **Vulnerability Name: -**

   SSOpenSSL_OP_NETSCAPE_REUSE_CIPHER_BUG Session Resume Ciphersuite Downgrade Issue

   Severity: - Medium

   Plugin: - 51892

   Description: - The remote host supports SSLv2 and therefore may be affected by a vulnerability that allows a cross-protocol Breitenbach padding oracle attack known as DROWN (Decrypting RSA with Obsolete and Weakened Encryption). This vulnerability exists due to a flaw in the Secure Sockets Layer Version 2 (SSLv2) implementation, and it allows captured TLS traffic to be decrypted. A man-in-the-middle attacker can exploit this to decrypt the TLS connection by utilizing previously captured traffic and weak cryptography along with a series of specially crafted connections to an SSLv2 server that uses the same private key.

   Solution: - To mitigate this vulnerability, administrators should disable the OpenSSL_OP_NETSCAPE_REUSE_CIPHER_BUG option in the OpenSSL configuration so that weak cipher suites cannot be reused during session resumption.

   Servers should be configured to enforce strong cipher suites and secure TLS protocols, preferably TLS 1.2 or TLS 1.3, and ensure that session resumption always follows proper cipher negotiation rules. Weak or outdated cipher suites should be removed from the server configuration.

   It is also recommended to update OpenSSL and related SSL/TLS libraries to the latest stable version, as newer versions include improved security controls and patches for known vulnerabilities.

   After applying the changes, administrators should perform security testing and vulnerability scanning using tools such as Nessus to confirm that the downgrade issue has been resolved. Regular security reviews and cryptographic configuration audits should also be conducted to maintain strong encryption standards across all systems.

   Business Impact: - This vulnerability occurs due to a cipher suite downgrade issue during SSL/TLS session resumption caused by the OpenSSL_OP_NETSCAPE_REUSE_CIPHER_BUG option. When this

configuration is enabled, the server may allow a previously negotiated weak cipher suite to be reused during a resumed SSL/TLS session.

An attacker performing a Man-in-the-Middle (MITM) attack may exploit this issue to force the connection to reuse a weaker cipher suite instead of negotiating a stronger and more secure one. As a result, the encryption strength of the communication session may be reduced, making it easier for attackers to analyze or decrypt sensitive traffic.

If exploited, this vulnerability may lead to exposure of sensitive data such as authentication credentials, session tokens, and confidential business information. It may also weaken the overall encryption posture of the organization, increasing the risk of data interception, unauthorized access, and compromise of secure communications. Over time, this can result in loss of customer trust, reputational damage, and potential compliance issues.

6. **Vulnerability Name: -** SSL Certificate Cannot Be Trusted

   Severity: - Medium

   Plugin: - 51192

   Description: - The server's X.509 certificate cannot be trusted. This situation can occur in three different ways, in which the chain of trust can be broken, as stated below :

   - First, the top of the certificate chain sent by the server might not be descended from a known public certificate authority. This can occur either when the top of the chain is an unrecognized, self-signed certificate, or when intermediate certificates are missing that would connect the top of the certificate chain to a known public certificate authority.

   - Second, the certificate chain may contain a certificate that is not valid at the time of the scan. This can occur either when the scan occurs before one of the certificate's 'notBefore' dates, or after one of the certificate's 'notAfter' dates.

- Third, the certificate chain may contain a signature that either didn't match the certificate's information or could not be verified. Bad signatures can be fixed by getting the certificate with the bad signature to be re-signed by its issuer. Signatures that could not be verified are the result of the certificate's issuer using a signing algorithm that Nessus either does not support or does not recognize.

If the remote host is a public host in production, any break in the chain makes it more difficult for users to verify the authenticity and identity of the web server. This could make it easier to carry out man-in-the-middle attacks against the remote host.

Solution: - To resolve this vulnerability, administrators should ensure that the server uses a valid SSL/TLS certificate issued by a trusted Certificate Authority (CA).

The full certificate chain, including all intermediate certificates, must be correctly installed on the server so that clients can verify the trust chain from the server certificate to a trusted root CA.

If the certificate is expired or not yet valid, it should be renewed or replaced with a new valid certificate. In cases where the certificate contains an invalid or unverifiable signature, the certificate should be reissued and properly signed by its trusted issuer.

Administrators should also regularly monitor certificate validity, expiration dates, and configuration settings to prevent similar issues in the future. After fixing the certificate configuration, a security scan using tools such as Nessus should be performed to confirm that the certificate chain is valid and trusted.

Business Impact: - This vulnerability indicates that the SSL/TLS certificate presented by the server cannot be trusted because the certificate chain is broken or invalid. When a certificate is not trusted, users and systems cannot properly verify the identity of the server they are connecting to.

If the certificate is self-signed, expired, not yet valid, or missing intermediate certificates, browsers and security tools may display warnings to users. This can reduce user confidence and may cause users to avoid accessing the website or application.

More importantly, an untrusted certificate increases the risk of Man-in-the-Middle (MITM) attacks, where an attacker may impersonate the legitimate server and intercept or modify sensitive communication. This could lead to exposure of confidential information such as login credentials, financial data, and personal user information.

For organizations running public-facing services, this issue can also lead to loss of customer trust, reputational damage, and possible compliance issues with security standards that require valid and trusted certificates.

7. **Vulnerability Name: -** TLS Version 1.0 Protocol Detection

   Severity: - Medium

   Plugin: - 104743

   Description: - The remote service accepts connections encrypted using TLS 1.0. TLS 1.0 has a number of cryptographic design flaws. Modern implementations of TLS 1.0 mitigate these problems, but newer versions of TLS like 1.2 and 1.3 are designed against these flaws.

   As of March 31, 2020, Endpoints that aren't enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

   PCI DSS v3.2 requires that TLS 1.0 be disabled entirely by June 30, 2018, except for POS POI terminals (and the SSL/TLS termination points to which they connect) that can be verified as not being susceptible to any known exploits.

   Solution: - To address this vulnerability, administrators should disable TLS 1.0 on all servers, applications, and network devices and ensure that only secure protocols such as TLS 1.2 and TLS 1.3 are enabled.

Server configurations should be updated to remove support for TLS 1.0 and enforce the use of strong encryption protocols and secure cipher suites. Additionally, weak or outdated cipher suites associated with older TLS versions should be disabled.

Organizations should also ensure that operating systems, web servers, and cryptographic libraries are updated to the latest versions to support modern TLS standards. After implementing these changes, administrators should perform vulnerability scans using security tools such as Nessus to verify that TLS 1.0 has been successfully disabled.

Regular security audits, patch management, and encryption configuration reviews should be conducted to maintain a strong security posture and ensure compliance with industry standards.

Business Impact: - The remote service supports TLS 1.0, which is an outdated cryptographic protocol with known security weaknesses. Although some modern implementations attempt to reduce these risks, TLS 1.0 still contains design flaws that make encrypted communications less secure compared to newer versions such as TLS 1.2 and TLS 1.3.

Attackers may exploit weaknesses in TLS 1.0 to perform Man-in-the-Middle (MITM) attacks, allowing them to intercept, monitor, or manipulate data transmitted between users and the server. This could potentially expose sensitive information such as login credentials, financial data, personal details, and confidential business communications.

In addition, major web browsers and technology vendors have deprecated support for TLS 1.0 since March 31, 2020, which can lead to compatibility issues with modern browsers and services. From a compliance perspective, PCI DSS v3.2 requires organizations to disable TLS 1.0, except in limited legacy scenarios. Failure to follow these standards may result in regulatory non-compliance, security risks, financial penalties, and reputational damage.

8. **Vulnerability Name: -** SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

   Severity: - Low

Plugin: - 83875

Description: - The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

Solution: - To remediate this vulnerability, administrators should replace weak Diffie-Hellman parameters with stronger ones, preferably using 2048-bit or higher modulus sizes.

Servers should be configured to support strong cipher suites and secure key exchange mechanisms, such as Elliptic Curve Diffie-Hellman (ECDHE), which provides better security and performance.

Additionally, system administrators should update SSL/TLS configurations, remove weak cipher suites, and ensure that modern TLS protocols (TLS 1.2 or TLS 1.3) are enabled. Updating web servers, cryptographic libraries, and operating systems to the latest secure versions is also recommended.

After implementing the changes, organizations should perform vulnerability scanning using tools such as Nessus to confirm that weak Diffie-Hellman parameters are no longer supported. Regular security configuration reviews and cryptographic updates should be maintained to ensure strong encryption practices.

Business Impact: - The remote server supports Diffie-Hellman (DH) key exchange parameters with a modulus size of 1024 bits or less, which is considered weak by modern cryptographic standards. This weakness is associated with the Logjam attack, where attackers may exploit weak Diffie-Hellman parameters to compromise the security of encrypted communications.

If an attacker has sufficient computational resources, they may perform cryptanalysis to calculate the shared secret used during the TLS handshake. Once the shared secret is discovered, the attacker could potentially decrypt sensitive information or manipulate the communication between the client and the server.

Although this vulnerability is rated as low severity, it still weakens the overall encryption strength of the system. If exploited, it may lead to exposure of confidential data, reduced integrity of secure sessions, and increased risk of interception attacks. Over time, weak cryptographic configurations may also lead to non-compliance with modern security best practices and standards.

9. **Vulnerability Name: -** SSLv3 Padding Oracle On Downgraded Legacy Encryption Vulnerability (POODLE)

   Severity: - Low

   Plugin: - 78479

   Description: - The remote host is affected by a man-in-the-middle (MitM) information disclosure vulnerability known as POODLE. The vulnerability is due to the way SSL 3.0 handles padding bytes when decrypting messages encrypted using block ciphers in cipher block chaining (CBC) mode. MitM attackers can decrypt a selected byte of a cipher text in as few as 256 tries if they are able to force a victim application to repeatedly send the same data over newly created SSL 3.0 connections.

   The TLS Fallback SCSV mechanism prevents 'version rollback' attacks without impacting legacy clients; however, it can only protect connections when the client and service support the mechanism. Sites that cannot disable SSLv3 immediately should enable this mechanism.

   This is a vulnerability in the SSLv3 specification, not in any particular SSL implementation. Disabling SSLv3 is the only way to completely mitigate the vulnerability.

   Solution: - To fully mitigate this vulnerability, administrators should disable SSLv3 on all servers and network devices and ensure that only modern secure protocols such as TLS 1.2 or TLS 1.3 are enabled.

   If disabling SSLv3 immediately is not possible due to legacy system requirements, organizations should implement the TLS_FALLBACK_SCSV mechanism, which helps prevent attackers from

forcing a downgrade attack. However, this should only be considered a temporary solution.

Administrators should also review the SSL/TLS configuration to remove weak cipher suites and outdated encryption protocols. Updating web servers, operating systems, and cryptographic libraries to the latest secure versions is recommended.

After implementing these changes, organizations should perform vulnerability scanning using security tools such as Nessus to confirm that SSLv3 is disabled and that the system is no longer vulnerable to the POODLE attack. Regular security audits and configuration reviews should also be conducted to maintain strong encryption practices.

Business Impact: - The remote system is vulnerable to the POODLE (Padding Oracle on Downgraded Legacy Encryption) attack, which affects the SSL 3.0 protocol. This vulnerability occurs due to improper handling of padding bytes in CBC (Cipher Block Chaining) encryption mode used by SSLv3.

A Man-in-the-Middle (MitM) attacker may exploit this vulnerability by forcing a connection downgrade from secure TLS versions to SSLv3. Once the connection is downgraded, the attacker can repeatedly manipulate encrypted messages and eventually decrypt small portions of the encrypted data, such as session cookies or authentication tokens.

If successful, attackers may gain unauthorized access to user sessions, confidential information, or sensitive communication data transmitted between the client and the server. Even though this vulnerability is rated as low severity, it still weakens the security of encrypted connections and increases the risk of data interception and session hijacking.

Additionally, the use of SSLv3 does not comply with modern security standards and industry best practices, which may lead to compliance issues and reduced trust in the organization's security posture.

10. **Vulnerability Name: -** SSL/TLS EXPORT_DHE <= 512-bit Export Cipher Suites Supported (Logjam)

    Severity: - Low

Plugin: - 83738

Description: - The remote host supports EXPORT_DHE cipher suites with keys less than or equal to 512 bits. Through cryptanalysis, a third party can find the shared secret in a short amount of time.

A man-in-the middle attacker may be able to downgrade the session to use EXPORT_DHE cipher suites. Thus, it is recommended to remove support for weak cipher suites.

Solution: - To mitigate this vulnerability, administrators should disable all EXPORT cipher suites, especially EXPORT_DHE cipher suites with 512-bit keys, from the SSL/TLS configuration on servers and network devices.

Servers should be configured to support only strong cipher suites and modern encryption standards, such as AES-based cipher suites with 128-bit or 256-bit keys and secure key exchange methods like ECDHE.

Additionally, organizations should ensure that only modern TLS protocols such as TLS 1.2 and TLS 1.3 are enabled, while outdated protocols and weak cryptographic algorithms are removed.

After implementing the configuration changes, administrators should perform vulnerability scanning using tools such as Nessus to confirm that weak export cipher suites are no longer supported. Regular security configuration reviews and patch management practices should also be followed to maintain strong encryption and prevent similar vulnerabilities in the future.

Business Impact: - The remote server supports EXPORT_DHE cipher suites with 512-bit keys, which are considered extremely weak by modern cryptographic standards. These export-grade cipher suites were originally designed to comply with historical export restrictions and provide very limited encryption strength.

An attacker performing a Man-in-the-Middle (MITM) attack may be able to force a connection downgrade so that the session uses these weak EXPORT_DHE cipher suites. Once the connection is downgraded, attackers with sufficient computational resources can break the weak 512-bit encryption and recover the shared secret used during the TLS handshake.

If exploited, attackers may be able to decrypt sensitive communication, access confidential information, or manipulate data transmitted between the client and server. Although the vulnerability is rated as low severity, the presence of weak cipher suites significantly weakens the security of encrypted connections and increases the risk of cryptographic attacks.

This vulnerability is also related to the Logjam attack, which targets weak Diffie-Hellman key exchange mechanisms. Supporting such outdated encryption methods may negatively impact the organization's security posture and compliance with modern security standards.

Stage – 3

Report

Title of the Project: -

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

1. Centralized Identity Management

Centralized Identity Management is one of the most important advantages of integrating Zscaler with Identity Providers (IdPs). In modern organizations, thousands of users need secure access to applications, networks, and cloud services. Managing separate login systems for each platform becomes complex and increases the risk of security issues.

By integrating Zscaler with identity providers like Azure AD, Okta, or Active Directory, all user identities are managed from a single central directory. The identity provider stores user credentials, roles, departments, and access permissions.

When a user tries to access the internet or a corporate application through Zscaler, the authentication request is redirected to the identity provider. The IdP verifies the user identity and then sends authentication confirmation back to Zscaler.

This centralized system provides several advantages:

• Easy management of user identities
• Reduced administrative workload
• Faster onboarding of new employees
• Immediate removal of access when an employee leaves
• Consistent access policies across the organization

For example, if a new employee joins the company, the IT administrator only needs to create the user account in Azure AD. The account will automatically apply to Zscaler authentication and access policies.

2. Single Sign-On (SSO) Authentication

Single Sign-On (SSO) allows users to access multiple services using only one login credential. When Zscaler is integrated with an Identity Provider, users can log in once and automatically gain access to all authorized resources without repeatedly entering their credentials.

This is usually implemented using authentication protocols such as **SAML (Security Assertion Markup Language)** or **OIDC (OpenID Connect)**.

The authentication process works as follows:

1. The user attempts to access the internet through Zscaler.
2. Zscaler redirects the login request to the identity provider (Azure AD, Okta, etc.).
3. The user enters their corporate credentials.
4. The identity provider verifies the user identity.
5. A secure authentication token is sent back to Zscaler.
6. Zscaler allows the user to access the network and applications.

Benefits of SSO include:

• Improved user experience
• Reduced password fatigue
• Fewer password reset requests
• Faster login process
• Stronger authentication security

For organizations with remote workers, SSO makes accessing secure networks simple and efficient.

3. Enhanced Security and Access Control

Security is the primary reason organizations integrate Zscaler with Identity Providers. The integration enables advanced identity-based security policies.

Instead of allowing access based only on IP addresses or devices, Zscaler can enforce policies based on user identity. This means security rules can be applied according to the user's role, department, location, or group membership.

For example:

• Finance department employees can access financial systems
• HR employees can access employee databases
• Interns may have limited internet access
• Remote employees may require stricter authentication

This approach follows the **Zero Trust security model**, where every user must verify their identity before accessing resources.

The benefits include:

• Strong protection against unauthorized access
• Reduced risk of insider threats
• Identity-based access control
• Better protection for sensitive data

If a user tries to access a restricted resource, Zscaler checks their identity information from the IdP and either grants or denies access.

4. Automated User Provisioning and Deprovisioning

Another important feature of integrating Zscaler with Identity Providers is automated user provisioning.

User provisioning refers to the process of creating, updating, and deleting user accounts across systems. Without automation, administrators must manually create accounts in multiple platforms, which is time-consuming and error-prone.

With IdP integration, user accounts and group information are automatically synchronized between the identity provider and Zscaler.

This process typically uses **SCIM (System for Cross-domain Identity Management)**.

The benefits include:

• Automatic creation of new user accounts
• Synchronization of user groups and roles
• Immediate removal of access for terminated employees
• Reduced manual administrative work
• Improved security compliance

For example:

When a new employee joins the marketing department, the administrator only creates the account in Active Directory. The system automatically syncs the user to Zscaler and applies the correct access policies.

Similarly, when an employee leaves the organization, disabling their account in the IdP immediately blocks their access to Zscaler services.

5. Multi-Factor Authentication (MFA) Support

Multi-Factor Authentication (MFA) provides an additional layer of security beyond passwords. When Zscaler is integrated with Identity Providers like Azure AD or Okta, organizations can enforce MFA for user authentication.

MFA requires users to verify their identity using two or more authentication factors.

Common MFA factors include:

• Something the user knows – password or PIN
• Something the user has – mobile phone or security token
• Something the user is – biometric verification (fingerprint or facial recognition)

The login process with MFA works as follows:

1. The user enters their username and password.
2. The identity provider verifies the credentials.
3. The system requests an additional verification factor.
4. The user enters a one-time password (OTP) or approves a push notification.
5. Access is granted to Zscaler services.

Benefits of MFA include:

• Strong protection against password theft
• Reduced risk of phishing attacks
• Improved identity verification
• Enhanced enterprise security

Even if an attacker steals a password, they cannot access the system without the second authentication factor.

6. Improved Monitoring, Logging, and Compliance

When Zscaler is integrated with identity providers, organizations gain improved visibility into user activity and authentication events.

Every login attempt, access request, and security event is recorded in logs generated by both Zscaler and the identity provider.

These logs help security teams monitor network activity and detect suspicious behavior.

Examples of logged events include:

• Successful login attempts
• Failed authentication attempts
• Access to restricted applications
• User activity on the network
• Location and device information

These logs are extremely valuable for:

• Security monitoring
• Threat detection
• Incident investigation
• Compliance reporting

Many organizations must comply with security standards such as:

• ISO 27001
• GDPR

• SOC 2
• HIPAA

Detailed audit logs help organizations demonstrate that proper security controls are implemented.

Security teams can also integrate these logs with **SIEM (Security Information and Event Management)** tools to detect anomalies and respond to potential cyber threats in real time.

Conclusion:

Stage 1:

Web Application Testing refers to the process of evaluating the security, functionality, and reliability of a web-based application to identify vulnerabilities, misconfigurations, and weaknesses that could be exploited by attackers. The primary goal of web application testing is to ensure that the application operates securely and that sensitive information exchanged between users and the server remains protected.

During Stage-1 of the assessment, web application testing helped in identifying security flaws related to encryption protocols, weak cipher suites, outdated software components, and certificate configuration issues. The testing process involved scanning the target system using automated vulnerability assessment tools to detect potential weaknesses in the server's SSL/TLS configuration and application dependencies.

From the results of this testing, it was observed that the application supports outdated security protocols such as SSLv2, SSLv3, and TLS 1.0, which are no longer considered secure according to modern cybersecurity standards. Additionally, the system was found to support weak cipher suites and weak Diffie-Hellman parameters, which reduce the strength of encryption used for secure communication.

Web application testing also revealed issues related to SSL certificate trust configuration and the use of outdated JavaScript libraries such as older versions of jQuery, which contain known Cross-Site Scripting (XSS) vulnerabilities. These weaknesses may allow attackers to exploit the application, intercept secure communication, or inject malicious scripts into the application.

Through this testing phase, it became clear that maintaining strong encryption protocols, updating software components, and following secure configuration practices are essential to protect web applications from potential cyber threats. Web application testing therefore plays a crucial role in detecting vulnerabilities early, improving the overall security posture of the system, and ensuring that the application complies with modern security standards and best practices.

Stage 2:

Stage-2 of the assessment involved analysing the Nessus vulnerability scan report to understand the security posture of the target system and identify the weaknesses that may expose the application to potential cyber threats. Nessus is a powerful vulnerability assessment tool that scans systems for known security vulnerabilities, misconfigurations, outdated protocols, and weak cryptographic settings.

From the Nessus report, it was observed that the target system contains multiple vulnerabilities primarily related to insecure SSL/TLS configurations, outdated encryption protocols, weak cipher suites, and outdated software libraries. These vulnerabilities highlight the presence of legacy security mechanisms that no longer meet modern cybersecurity standards.

One of the major findings from the report is that the system still supports deprecated encryption protocols such as SSLv2, SSLv3, and TLS 1.0. These protocols have known security weaknesses and are vulnerable to attacks such as POODLE and DROWN, which can allow attackers to decrypt encrypted communication or force a downgrade to weaker encryption protocols. The presence of these outdated protocols significantly increases the risk of Man-in-the-Middle (MITM) attacks, where attackers may intercept and manipulate secure communications between users and the server.

The report also identified the use of weak and medium-strength cipher suites, including EXPORT-grade encryption and weak Diffie-Hellman parameters. Vulnerabilities such as Logjam demonstrate that weak key exchange mechanisms can allow attackers to break encryption by performing cryptographic attacks. If exploited, attackers may recover encryption keys, decrypt sensitive data, or compromise the integrity of secure connections.

Another important observation from the Nessus report is the presence of SSL certificate trust issues, where the certificate chain could not be fully validated. This can occur when certificates are expired, self-signed, improperly configured, or missing intermediate certificates. Untrusted certificates may cause browsers to display security warnings and increase the risk of server impersonation attacks, where attackers may attempt to masquerade as the legitimate server.

In addition to cryptographic vulnerabilities, the report also detected the use of an outdated version of the jQuery library, which is vulnerable to multiple Cross-Site Scripting (XSS) attacks. These vulnerabilities may allow attackers to inject malicious scripts into web pages, potentially leading to session hijacking, credential theft, or unauthorized actions performed on behalf of legitimate users.

Overall, the Nessus report clearly indicates that the system requires improvements in its security configuration, encryption standards, and software update practices. Many of the vulnerabilities identified are related to outdated technologies and weak cryptographic configurations that should be replaced with modern security mechanisms.

To improve the security posture of the system, it is recommended to disable outdated protocols such as SSLv2, SSLv3, and TLS 1.0, remove weak cipher suites, implement stronger encryption protocols such as TLS 1.2 or TLS 1.3, update vulnerable software libraries, and

properly configure trusted SSL certificates. Regular vulnerability scanning and security assessments should also be conducted to ensure that newly discovered vulnerabilities are detected and mitigated promptly.

By addressing the vulnerabilities identified in the Nessus report, the organization can significantly reduce the attack surface, strengthen the confidentiality and integrity of communications, and improve compliance with modern cybersecurity standards and best practices.

Stage 3:

Stage-3 of the assessment focused on reviewing the remediation strategies and security recommendations based on the vulnerabilities identified in the Nessus vulnerability assessment report. The main objective of this stage was to understand how the detected security weaknesses can be mitigated and what improvements should be implemented to strengthen the overall security posture of the system.

From the analysis of the document, it was observed that the majority of the vulnerabilities detected during the scanning process were related to weak SSL/TLS configurations, outdated encryption protocols, insecure cipher suites, and outdated application libraries. These vulnerabilities indicate that the system was configured with legacy security mechanisms that are no longer considered secure according to modern cybersecurity standards.

During this stage, emphasis was placed on implementing secure configuration practices and stronger encryption mechanisms. One of the most important recommendations is to disable outdated protocols such as SSLv2, SSLv3, and TLS 1.0, as these protocols contain several cryptographic weaknesses and are vulnerable to attacks such as POODLE, DROWN, and downgrade attacks. Replacing these protocols with secure versions such as TLS 1.2 or TLS 1.3 significantly improves the confidentiality and integrity of communications between clients and the server.

Another key improvement discussed in the document is the removal of weak and export-grade cipher suites. Weak cipher suites reduce the effectiveness of encryption and can allow attackers to perform cryptographic attacks such as Logjam, which targets weak Diffie-Hellman parameters. To mitigate this risk, the system should be configured to support only strong cipher suites using modern encryption algorithms such as AES with secure key exchange mechanisms like ECDHE.

The document also highlights the importance of updating outdated software components and third-party libraries, such as upgrading vulnerable versions of jQuery that contain multiple Cross-Site Scripting (XSS) vulnerabilities. Keeping application dependencies updated ensures that known security flaws are patched and reduces the attack surface of the web application.

Additionally, proper SSL certificate management is required to ensure that certificates are issued by trusted Certificate Authorities and that the certificate chain is correctly configured. Valid and trusted certificates help users verify the authenticity of the server and prevent Man-in-the-Middle attacks.

Overall, Stage-3 emphasizes the implementation of remediation measures and best security practices to address the vulnerabilities identified in earlier stages. By applying the recommended security improvements, the system can achieve a stronger security posture, reduce the risk of cyberattacks, and ensure compliance with modern cybersecurity standards.

In conclusion, Stage-3 demonstrates that continuous vulnerability management, regular security updates, strong encryption configurations, and proactive security monitoring are essential for maintaining a secure and reliable web application environment. Implementing these recommendations will significantly reduce potential security risks and enhance the overall protection of the system and its data.

Future Scope:

Stage 1:

The first stage of the project focused on understanding the fundamentals of web application testing and identifying vulnerabilities present in the system through initial security analysis. Although the current testing process helped in identifying several security weaknesses, there are multiple opportunities to further enhance the effectiveness and coverage of web application security testing in the future.

In the future, web application testing can be expanded by implementing more advanced security testing techniques such as penetration testing, dynamic application security testing (DAST), static application security testing (SAST), and interactive application security testing (IAST). These techniques can help in identifying deeper vulnerabilities that may not be detected through basic scanning methods.

Another important future improvement is the integration of automated security testing tools into the software development lifecycle (SDLC). By incorporating security testing in early development stages, organizations can detect vulnerabilities during development rather than after deployment, which significantly reduces security risks and remediation costs.

Future work may also include continuous security monitoring and regular vulnerability assessments to ensure that new vulnerabilities are detected and addressed promptly. As web technologies and cyber threats evolve rapidly, regular testing is essential to maintain a strong security posture.

Additionally, future web application testing can focus on testing complex attack vectors such as authentication bypass, session management flaws, API security vulnerabilities, and business logic flaws. These areas often require advanced testing methodologies and manual security analysis.

Another potential enhancement is the implementation of secure coding practices and developer security training, which can help developers build applications that are resistant to common vulnerabilities such as Cross-Site Scripting (XSS), SQL Injection, and insecure configurations.

In conclusion, the future scope of Stage-1 involves expanding web application testing methodologies, adopting advanced security tools, integrating security into the development

process, and continuously monitoring the application to ensure long-term protection against evolving cyber threats.

Stage 2:

Stage-2 of the project focused on analyzing the vulnerability assessment results generated by the Nessus scanning tool. The report provided valuable insights into the security weaknesses present in the system, particularly related to SSL/TLS configurations, weak cipher suites, outdated protocols, and vulnerable software components. While the current analysis helped in identifying major vulnerabilities, there are several opportunities to further improve the vulnerability assessment process in the future.

In the future, vulnerability scanning can be expanded by using multiple security assessment tools along with Nessus, such as OpenVAS, Burp Suite, Nikto, and Nmap vulnerability scripts. Using multiple tools allows security analysts to perform cross-verification of results and detect vulnerabilities that may not be identified by a single scanning tool.

Another important improvement is the implementation of continuous vulnerability monitoring instead of performing occasional scans. Continuous scanning helps organizations identify newly discovered vulnerabilities as soon as they appear and enables faster remediation before attackers can exploit them.

Future work may also involve deeper analysis of vulnerability reports, including risk prioritization, impact analysis, and vulnerability correlation. By prioritizing vulnerabilities based on severity, exploitability, and business impact, organizations can focus on fixing the most critical issues first.

Additionally, organizations can integrate automated vulnerability management systems that track vulnerabilities, assign remediation tasks, and monitor the progress of security fixes. This approach improves the efficiency of vulnerability management and ensures that security issues are resolved systematically.

Overall, the future scope of Stage-2 lies in improving vulnerability analysis through advanced scanning techniques, integration of multiple security tools, automated vulnerability management, and continuous monitoring, which will help strengthen the overall security posture of the system.

Stage 3:

Stage-3 of the project focused on understanding the remediation strategies required to address the vulnerabilities identified during the vulnerability assessment process. This stage emphasized improving the system's security configuration by disabling outdated protocols, removing weak cipher suites, updating vulnerable software components, and implementing stronger encryption standards.

In the future, organizations can enhance the remediation process by implementing automated patch management systems that ensure software components, operating systems, and

application libraries are regularly updated with the latest security patches. Automated patching helps reduce the risk of vulnerabilities caused by outdated software.

Another important future improvement is the adoption of secure configuration management and security hardening practices for servers and web applications. This includes enforcing strong encryption standards, implementing secure authentication mechanisms, and regularly reviewing system configurations to eliminate potential weaknesses.

Future work can also include the implementation of advanced security mechanisms such as Web Application Firewalls (WAF), Intrusion Detection Systems (IDS), and Intrusion Prevention Systems (IPS). These security solutions help monitor network traffic and detect suspicious activities that may indicate potential cyber attacks.

Additionally, organizations can adopt DevSecOps practices, where security is integrated directly into the development and deployment processes. This approach ensures that security checks are performed continuously throughout the development lifecycle, reducing the chances of vulnerabilities appearing in production systems.

In conclusion, the future scope of Stage-3 focuses on implementing continuous security improvement strategies, automated patch management, advanced security monitoring solutions, and proactive vulnerability management. These measures will help maintain a secure web application environment and protect the system against evolving cyber threats.

Topic Explored: -

- Okta
- Integration
- Zero Trust

Tools Explored: -

- Zscaler
- Nessus
- Owasp Top 10
- http://zero.webappsecurity.com/